Machine Learning for Finance

A Machine Learning Algorithm for Finite-Horizon Stochastic Control Problems in Economics (2024)

BAGOT Rémi, CHARDIGNY Lola, LETACONNOUX Lauryn

April 2026

Contents

1 Abstract **2**

2 Why MMCC is introduced **2**

3 The MMCC Algorithm **2**
3.1 MMCC Algorithm in time-separable case 3
3.2 MMCC algorithm in the non time-separable case 4

4 The Adam Algorithm [2] **5**

5 The Application **7**
5.1 The control problem 7
5.2 The discretized control problem 8
5.3 For the implementation of the MMCC algorithm 8
5.4 Numerical Results 9

6 Conclusion **11**

1 Abstract

We apply the MMCC algorithm to a challenging finite-horizon, path-dependent stochastic control problem, where the state dimension, although not high-dimensional in the extreme sense, is already rich enough to make classical Bellman-based dynamic programming approaches rapidly impractical. In particular, we study the dynamic hedging of an Asian option under a Heston stochastic volatility model with proportional transaction costs. The Python implementation of this project is available on GitHub:
`https://github.com/laurynletaconnoux/option-princing-with-mmcc-algorithm`

2 Why MMCC is introduced

The article [1] introduces the MMCC algorithm to address three major difficulties that arise in stochastic control problems. First, many relevant problems in economics and finance are formulated over a finite-horizon and are time-inhomogeneous, so that the optimal policy depends explicitly on the date and the problem becomes more difficult than in the stationary infinite-horizon setting. Second, these problems are often high-dimensional and may involve complex state dynamics, which makes classical numerical methods quickly impractical because of the curse of dimensionality. Third, when the utility function or the objective is not time separable, a Bellman equation may no longer be available, which weakens the relevance of standard dynamic programming approaches.
In this setting, MMCC is proposed as a simulation-based algorithm that can handle finite-horizon, time-inhomogeneous, and high-dimensional stochastic control problems without relying on the Bellman equation. The idea is to approximate the optimal policies directly with neural networks and to update them sequentially in a backward direction. The authors emphasize the fact that the method results in a monotonic improvement of the objective function at each iteration, which supports good convergence properties.

3 The MMCC Algorithm

The Machine Learning Monte Carlo Control (MMCC) algorithm is an iterative procedure designed to solve stochastic control problems by combining Monte Carlo simulation and neural network optimization.
Before detailing the algorithm, we introduce the main notations. We consider a discrete-time stochastic control problem over a finite horizon T. At each date t, the system is described by a state variable s_t and a control variable c_t. The initial control is denoted by c_0, while for $t \geq 1$ the control is represented by a neural-network policy $c(t, s_t, \theta_t)$, where θ_t is the vector of trainable network parameters at date t. The state evolves according to the transition function ψ_{t+1}, driven by an exogenous random shock z_{t+1}. In the algorithm, the superscript k denotes the MMCC iteration, the index l labels the simulated trajectory, and the index i labels the mini-batch.

The first version of the MMCC algorithm is designed to solve the finite-horizon stochastic control problem with time-separable objective introduced as Problem (3) in the article, that is,

$$\max_{(c_0,\theta_1,\ldots,\theta_{T-1})\in\Theta} \mathbb{E}_0 \left[\sum_{t=0}^{T-1} u_{t+1}(s_{t+1}, s_t, c_t) \,\middle|\, c_0, \theta_1, \ldots, \theta_{T-1} \right]$$

subject to

$$c_t = c(t, s_t, \theta_t), \qquad s_{t+1} = \psi_{t+1}(s_t, c_t, z_{t+1}), \qquad 0 \leq t \leq T-1.$$

3.1 MMCC Algorithm in time-separable case

In this case, we consider a time-separable utility function u.

Initialization: Set $k = 1$ and initialize the vector of control parameters

$$x^0 = (c_0^0, \theta_1^0, \theta_2^0, \ldots, \theta_{T-1}^0).$$

The control policy is parameterized by a neural network of the form

$$c(t, s_t, \theta_t) = \sigma_K \circ T_K \circ \cdots \circ \sigma_1 \circ T_1.$$

Iteration: (k-th iteration) Repeat the following steps until a stopping criterion is satisfied.

Step A (Monte Carlo simulation) Given the current parameter vector

$$x^{k-1} = (c_0^{k-1}, \theta_1^{k-1}, \ldots, \theta_{T-1}^{k-1}),$$

simulate N i.i.d. sample paths of the state process.
For each trajectory $l = 1, \ldots, N$, set $s_{0,l}^k = s_0$ and recursively compute

$$c_{t,l}^k = c(t, s_{t,l}^k, \theta_t^{k-1}), \quad s_{t+1,l}^k = \psi_{t+1}(s_{t,l}^k, c_{t,l}^k, z_{t+1,l}), \quad t = 0, \ldots, T-1.$$

The N simulated paths are then partitioned into m mini-batches of size b, such that $N = mb$.

Step B (Backward optimization of the policy parameters) For $t = T-1, T-2, \ldots, 1$, the parameter θ_t is updated by solving a sequence of stochastic optimization problems.
For each minibatch $i = 1, \ldots, m$, we maximize the following expectation:

$$\begin{aligned}
&\max_{\theta_t} \quad \mathbb{E}_0 \left[\sum_{j=t}^{T-1} u_{j+1}(s_{j+1}, s_j, c_j) \;\middle|\; c_0^{k-1}, \theta_1^{k-1}, \ldots, \theta_{t-1}^{k-1}, \theta_t, \theta_{t+1}^k, \ldots, \theta_{T-1}^k \right] \\
&= \max_{\theta_t} \quad \mathbb{E}_0 \Bigg\{ \frac{1}{b} \sum_{l=1}^{b} \Bigg[u_{t+1}\big(s_{t+1,l,i}^k(\theta_t),\, s_{t,l,i}^k,\, c_{t,l,i}^k(\theta_t)\big) \\
&\qquad\qquad + \sum_{j=t+1}^{T-1} u_{j+1}\big(s_{j+1,l,i}^k(\theta_t),\, s_{j,l,i}^k(\theta_t),\, c_{j,l,i}^k(\theta_t)\big) \Bigg] \Bigg\}.
\end{aligned} \tag{1}$$

where $c_{t,l,i}^k(\theta_t) = c(t, s_{t,l,i}^k, \theta_t)$ and the sequence

$$\big(s_{t+1,l,i}^k(\theta_t),\, c_{t+1,l,i}^k(\theta_t),\, \ldots,\, s_{T-1,l,i}^k(\theta_t),\, c_{T-1,l,i}^k(\theta_t),\, s_{T,l,i}^k(\theta_t)\big)$$

is the simulated trajectory starting from $s_{t,l,i}^k$ and follows the control parameters $\theta_t, \theta_{t+1}^k, \ldots, \theta_{T-1}^k$:

- $s_{t+1,l,i}^k(\theta_t) = \psi(s_{t,l,i}^k, c_{t,l,i}^k(\theta_t), z_{t+1,l,i})$
- $c_{t+1,l,i}^k(\theta_t) = c(t+1, s_{t+1,l,i}^k(\theta_t), \theta_{t+1}^k)$
- $s_{t+2,l,i}^k(\theta_t) = \psi(s_{t+1,l,i}^k(\theta_t), c_{t+1,l,i}^k(\theta_t), z_{t+2,l,i})$
- $c_{t+2,l,i}^k(\theta_t) = c(t+2, s_{t+2,l,i}^k(\theta_t), \theta_{t+2}^k)$
- $\ldots$

The parameter θ_t is updated using the **Adam optimization algorithm**. After processing all mini-batches, we obtain the optimal updated parameter θ_t^k.
*The total computational cost of **Step A** is $\mathcal{O}(m(T-t+1))$.*

Step C (Optimization of the initial control) Finally, the initial control c_0 is updated using a similar Monte Carlo approximation.
For each minibatch $i = 1, \ldots, m$, we consider the following conditional expectation:

$$\begin{aligned} &\max_{c_0} \quad \mathbb{E}_0 \left[\sum_{j=0}^{T-1} u_{j+1}(s_{j+1}, s_j, c_j) \,\middle|\, c_0, \theta_1^k, \ldots, \theta_{T-1}^k \right] \\ &= \max_{c_0} \quad \mathbb{E}_0 \left\{ \frac{1}{b} \sum_{l=1}^{b} \left[u_1\big(s_{1,l,i}^k(c_0),\, s_0,\, c_0\big) + \sum_{j=1}^{T-1} u_{j+1}\big(s_{j+1,l,i}^k(c_0),\, s_{j,l,i}^k(c_0),\, c_{j,l,i}^k(c_0)\big) \right] \right\}. \end{aligned} \tag{2}$$

Where

$$\left\{ s_{1,l,i}^k(c_0),\, c_{1,l,i}^k(c_0),\, \ldots,\, s_{T-1,l,i}^k(c_0),\, c_{T-1,l,i}^k(c_0),\, s_{T,l,i}^k(c_0) \right\}_{l=1}^{b}$$

is a sequence of sample paths simulated starting from s_0 and follows the control parameters: $(c_0, \theta_1^k, \ldots, \theta_{T-1}^k)$.
The control c_0 is updated using the Adam algorithm, yielding c_0^k.
*The total computational cost of **Step C** is $\mathcal{O}(mT)$.*

Output from the iteration: The parameter vector is updated as

$$x^{k-1} \leftarrow x^k = (c_0^k, \theta_1^k, \theta_2^k, \ldots, \theta_{T-1}^k),$$

and the iteration counter is incremented, $k \leftarrow k + 1$.
*The total computational cost of each iteration of **MMCC algorithm** is $\mathcal{O}\big(\frac{m}{2}(T^2 - T)\big) = \mathcal{O}(mT^2)$.*

The second version of the MMCC algorithm is designed to solve the more general finite-horizon stochastic control problem introduced as Problem (5) in the article, in which the objective function may depend on the whole trajectory and therefore need not be time-separable:

$$\max_{(c_0, \theta_1, \ldots, \theta_{T-1}) \in \Theta} \mathbb{E}_0 \left[u(s_0, c_0, s_1, c_1, \ldots, s_{T-1}, c_{T-1}, s_T) \,|\, c_0, \theta_1, \ldots, \theta_{T-1} \right]$$

subject to

$$c_t = c(t, s_t, \theta_t), \qquad s_{t+1} = \psi_{t+1}(s_t, c_t, z_{t+1}), \qquad 0 \le t \le T - 1.$$

3.2 MMCC algorithm in the non time-separable case

As pointed out in the article, the second algorithm addresses a more general class of stochastic control problems, for which the objective function need not be time-separable. Nevertheless, the MMCC algorithm remains largely unchanged in the non time-separable case. The only difference lies in the expectations that are maximized in **Step B** and **Step C**.
In this case, the utility function u depends on the entire trajectory $(s_0, c_0, \ldots, s_T)$, rather than only on the future trajectory starting from s_t.

Step B (Backward optimization of the policy parameters) For $t = T-1, T-2, \ldots, 1$, the parameter θ_t is updated by solving a sequence of stochastic optimization problems.
For each minibatch $i = 1, \ldots, m$, we maximize the following expectation:

$$\begin{aligned} &\max_{\theta_t} \quad \mathbb{E}_0 \left[u(s_0, c_0, s_1, c_1, \ldots, s_{T-1}, c_{T-1}, s_T, c_T) \,\middle|\, c_0^{k-1}, \theta_1^{k-1}, \ldots, \theta_{t-1}^{k-1}, \theta_t, \theta_{t+1}^k, \ldots, \theta_{T-1}^k \right] \\ &= \max_{\theta_t} \quad \mathbb{E}_0 \left\{ \frac{1}{b} \sum_{l=1}^{b} \left[u\big(s_{0,l,i}^k, c_{0,l,i}^k, \ldots, s_{t,l,i}^k, c_{t,l,i}^k(\theta_t), s_{t+1,l,i}^k(\theta_t), c_{t+1,l,i}^k(\theta_t), \ldots, s_{T,l,i}^k(\theta_t), c_{T,l,i}^k(\theta_t)\big) \right] \right\}. \end{aligned} \tag{3}$$

where $c^k_{t,l,i}(\theta_t) = c(t, s^k_{t,l,i}, \theta_t)$ and the sequence

$$\big(s^k_{t+1,l,i}(\theta_t),\ c^k_{t+1,l,i}(\theta_t),\ \ldots,\ s^k_{T-1,l,i}(\theta_t),\ c^k_{T-1,l,i}(\theta_t),\ s^k_{T,l,i}(\theta_t)\big)$$

is the simulated trajectory starting from $s^k_{t,l,i}$ and follows the control parameters $\theta_t, \theta^k_{t+1}, \ldots, \theta^k_{T-1}$:

- $s^k_{t+1,l,i}(\theta_t) = \psi(s^k_{t,l,i}, c^k_{t,l,i}(\theta_t), z_{t+1,l,i})$
- $c^k_{t+1,l,i}(\theta_t) = c(t+1, s^k_{t+1,l,i}(\theta_t), \theta^k_{t+1})$
- $s^k_{t+2,l,i}(\theta_t) = \psi(s^k_{t+1,l,i}(\theta_t), c^k_{t+1,l,i}(\theta_t), z_{t+2,l,i})$
- $c^k_{t+2,l,i}(\theta_t) = c(t+2, s^k_{t+2,l,i}(\theta_t), \theta^k_{t+2})$
- $\ldots$

$s^k_{0,l,i} = s_0$ and the sequence

$$\big(c^k_{0,l,i}, s^k_{1,l,i}, \ldots, c^k_{t,l,i}, s^k_{t,l,i},\big)$$

is already simulated in the **Step A** and follows the control parameters $\theta^{k-1}_1, \theta^{k-1}_2, \ldots, \theta^{k-1}_{t-1}$.
The parameter θ_t is updated using the **Adam optimization algorithm**. After processing all mini-batches, we obtain the optimal updated parameter θ^k_t.

Step C (Optimization of the initial control) Finally, the initial control c_0 is updated using a similar Monte Carlo approximation.
For each minibatch $i = 1, \ldots, m$, we consider the following conditional expectation:

$$\begin{aligned} &\max_{c_0} \quad \mathbb{E}_0\left[u(s_0, c_0, s_1, c_1, \ldots, s_{T-1}, c_{T-1}, s_T, c_T) \;\middle|\; c_0, \theta^k_1, \ldots, \theta^k_{T-1}\right] \\ &= \max_{c_0} \quad \mathbb{E}_0\left\{\frac{1}{b}\sum_{l=1}^{b}\left[u\big(s^k_{0,l,i}, c_0, s^k_{1,l,i}(c_0), c^k_{1,l,i}(c_0), \ldots, s^k_{T,l,i}(c_0), c^k_{T,l,i}(c_0)\big)\right]\right\}. \end{aligned} \tag{4}$$

Where $s^k_{0,l,i} = s_0$ and

$$\left\{s^k_{1,l,i}(c_0),\ c^k_{1,l,i}(c_0),\ \ldots,\ s^k_{T-1,l,i}(c_0),\ c^k_{T-1,l,i}(c_0),\ s^k_{T,l,i}(c_0)\right\}_{l=1}^{b}$$

is a sequence of sample paths simulated starting from s_0 and follows the control parameters: $(c_0, \theta^k_1, \ldots, \theta^k_{T-1})$.
The control c_0 is updated using the Adam algorithm, yielding c^k_0.

4 The Adam Algorithm [2]

Adam (Adaptive Moment Estimation) is an algorithm for stochastic optimization.
A common optimization problem in machine learning is

$$\min_{\theta\in\Theta} L_n(\theta) = \frac{1}{n}\sum_{i=1}^{n}\ell(Z_i, \theta).$$

Classical gradient descent has a cost of $\mathcal{O}(n)$ per iteration, and it is not suitable when data arrive in a continuous stream.
The idea of stochastic optimization is to use a subsample at each iteration, which leads in practice to convergence to a good optimum.

Stochastic Gradient Descent (SGD):

$$\theta^{(k+1)} = \theta^{(k)} - \alpha_k \nabla \ell(Z_{i_k}, \theta^{(k)})$$

where i_k is chosen uniformly at random in $\{1, \ldots, n\}$. The cost per iteration is $\mathcal{O}(1)$ instead of $\mathcal{O}(n)$, but the method is noisy and can be unstable.

SGD with minibatch:

$$\bigcup_{k=1}^{K} B_k = [n], \quad \#B_k = m = \frac{n}{K}$$

$$\theta^{(k+1)} = \theta^{(k)} - \alpha_k \frac{1}{m} \sum_{j \in B_k} \nabla \ell(Z_j, \theta^{(k)})$$

This provides a better trade-off between variance and efficiency. The cost per iteration is $\mathcal{O}(m)$.

Momentum:

$$v^{(k+1)} = \beta v^{(k)} + (1-\beta) \nabla \ell(Z_{i_k}, \theta^{(k)})$$
$$\theta^{(k+1)} = \theta^{(k)} - \alpha_k v^{(k+1)}$$

This method accumulates gradients from previous iterations. It reduces oscillations and accelerates convergence in narrow valleys.

RMSProp:

$$g^{(k+1)} = \gamma g^{(k)} + (1-\gamma) \left(\nabla \ell(Z_{i_k}, \theta^{(k)}) \right)^2$$
$$\theta^{(k+1)} = \theta^{(k)} - \frac{\alpha_k}{\sqrt{g^{(k+1)}} + \varepsilon} \nabla \ell(Z_{i_k}, \theta^{(k)})$$

This method adapts the learning rate for each coordinate.

Adaptive Moment Estimation (Adam): The Adam algorithm combines momentum and adaptive learning rates.

$$m^{(k+1)} = \beta_1 m^{(k)} + (1-\beta_1) \nabla \ell(Z_{i_k}, \theta^{(k)})$$
$$v^{(k+1)} = \beta_2 v^{(k)} + (1-\beta_2) \left(\nabla \ell(Z_{i_k}, \theta^{(k)}) \right)^2$$
$$\hat{m}^{(k+1)} = \frac{m^{(k+1)}}{1-\beta_1^{k+1}}, \quad \hat{v}^{(k+1)} = \frac{v^{(k+1)}}{1-\beta_2^{k+1}}$$
$$\theta^{(k+1)} = \theta^{(k)} - \alpha_k \frac{\hat{m}^{(k+1)}}{\sqrt{\hat{v}^{(k+1)}} + \varepsilon}$$

Adam can also be combined with mini-batches.

5 The Application

In order to illustrate the MMCC algorithm on a finance problem, we consider the dynamic hedging of an Asian option under stochastic volatility with proportional transaction costs. This is a natural application for MMCC, since it combines a finite-horizon setting, a path-dependent payoff, an augmented state representation, and market frictions, making standard dynamic programming approaches difficult to implement.

5.1 The control problem

We consider a financial market with a risky asset S_t, whose volatility follows the Heston stochastic volatility model: [3]:

$$dS_t = rS_t\,dt + \sqrt{V_t}\,S_t\,dW_t, \quad S_0 = s_0 \tag{5}$$

$$dV_t = \kappa(\theta - V_t)\,dt + \xi\sqrt{V_t}\left(\rho\,dW_t + \sqrt{1-\rho^2}\,d\widetilde{W}_t\right), \quad V_0 = v_0 \tag{6}$$

where r is the risk-free rate, $\kappa > 0$ is the mean-reversion speed, $\theta > 0$ is the long-run variance, $\xi > 0$ is the volatility of variance, W_t and $\widetilde{W}_t$ are two independent standard Brownian motions. We consider an Asian call option with strike K, whose payoff is

$$g(\bar{S}_T) = \left(\frac{1}{T}\int_0^T S_t\,dt - K\right)^+, \qquad \bar{S}_T = \frac{1}{T}\int_0^T S_t\,dt.$$

To embed the path dependence into the state vector, we introduce the auxiliary process:

$$A_0 = 0, \qquad dA_t = S_t\,dt,$$

so that

$$\bar{S}_T = \frac{A_T}{T}.$$

Let $\lambda > 0$ be the proportional transaction cost. Let Q_t be the stock position and B_t the cash account. Let $Q_0 = q_0$ be the initial stock position.
When trading dQ_t at time t, the transaction cost is

$$\Lambda_t = \lambda S_t\,|dQ_t|.$$

The cash account evolves as

$$dB_t = rB_t\,dt - S_t\,dQ_t - \Lambda_t, \quad B_0 = b_0.$$

The portfolio dynamics are

$$dY_t = dB_t + Q_t dS_t, \quad Y_0 = y.$$

At maturity, the position is liquidated:

$$Y_T = B_T + Q_T S_T - \Lambda_T.$$

We consider the following stochastic control problem, we define the objective we define the objective as minimizing the terminal quadratic hedging error:

$$\min_{y_0, Q(.)} \mathbb{E}\Big[\Big(Y_T - g(\bar{S}_T)\Big)^2\Big].$$

5.2 The discretized control problem

We discretize $[0,T]$ into $0 = t_0 < t_1 < \ldots < t_N = T$. The discretized control problem is

$$\min_{(y_0, q_0, \theta_{t_1}, \ldots, \theta_{t_{N-1}})} \mathbb{E}\Big[\Big(Y_T - g(\bar{S}_T)\Big)^2\Big]$$

where

$$g(\bar{S}_T) = \big(\frac{1}{N+1}\sum_{n=0}^{N} S_{t_n} - K\big)^+$$

subject to the dynamics

$$\begin{aligned}
&S_{t_0} = S_0, \quad V_{t_0} = v_0, \quad A_{t_0} = 0, \quad B_{t_0} = Y_{t_0} = y_0, \quad Q_{t_0} = q_0,\\
&Q_{t_n} = c(t_n, S_{t_n}, V_{t_n}, A_{t_n}, B_{t_n}, Y_{t_n}, \theta_{t_n}),\\
&S_{t_{n+1}} = S_{t_n} \exp\left(\left(r - \frac{1}{2}V_{t_n}^+\right)(t_{n+1} - t_n) + \sqrt{V_{t_n}^+}\,(W_{t_{n+1}} - W_{t_n})\right),\\
&V_{t_{n+1}} = V_{t_n} + \kappa(\theta - V_{t_n}^+)(t_{n+1} - t_n) + \xi\sqrt{V_{t_n}^+}\left(\rho\,(W_{t_{n+1}} - W_{t_n}) + \sqrt{1-\rho^2}\,(\widetilde{W}_{t_{n+1}} - \widetilde{W}_{t_n})\right),\\
&A_{t_{n+1}} = A_{t_n} + S_{t_{n+1}}(t_{n+1} - t_n),\\
&B_{t_{n+1}} = e^{r(t_{n+1} - t_n)}\left(B_{t_n} - (Q_{t_n} - Q_{t_{n-1}})S_{t_n} - \lambda S_{t_n}|Q_{t_n} - Q_{t_{n-1}}|\right),\\
&Y_{t_{n+1}} = Y_{t_n} + (B_{t_{n+1}} - B_{t_n}) + Q_{t_n}(S_{t_{n+1}} - S_{t_n}),\\
&Y_{t_N} = B_{t_N} + Q_{t_{N-1}}S_{t_N} - \lambda S_{t_N}|Q_{t_{N-1}}|.
\end{aligned}$$

The goal is to determine the initial capital y_0, which can be interpreted as an approximation of the option price at time t_0, the initial stock position q_0, which approximates the optimal initial hedge, and a family of neural-network policies that determine the hedging strategy at each date t_n, for $n = 1, \ldots, N-1$. In fact, the control

$$c(t_n, S_{t_n}, V_{t_n}, A_{t_n}, Q_{t_{n-1}}, B_{t_n}, \theta_{t_n})$$

is used as a neural network approximation of the optimal hedging strategy at time t_n.

5.3 For the implementation of the MMCC algorithm

The global vector of control parameters is

$$x = (y_0, q_0, \theta_{t_1}, \ldots, \theta_{t_{N-1}}), \qquad c_0 = (y_0, q_0)$$

For each date t_n, $n = 1, \ldots, N-1$, the state vector is

$$s_n = (S_{t_n}, V_{t_n}, A_{t_n}, Q_{t_{n-1}}, B_{t_n})$$

The rebalancing decision is:

$$Q_{t_n} = c(t_n, s_n, \theta_{t_n})$$

The transition from s_n to s_{n+1} is therefore given by

$$s_{n+1} = \psi_{n+1}(s_n, Q_{t_n}, z_{n+1}),$$

where $z_{n+1} \in \mathbb{R}^{n_z}$ is a random vector denoting the random shock in the $(t+1)$-th period, and ψ_{n+1} is the state evolution function.

5.4 Numerical Results

Neural Network Architecture: The neural network for $c(t_n, s_n, \theta_{t_n})$ has four layers: an input layer with 5 neurons, two hidden layers with 70 neurons each, and an output layer with 1 neuron.
The nonlinear activation function used in the hidden layers is the rectified linear unit (ReLU), which is chosen for its simplicity and its ability to mitigate the vanishing gradient problem, leading to efficient and stable training.
The inputs are normalized to have approximately the same scale (around 1), which significantly improves the stability and efficiency of the algorithm.

Numerical Setup: The initial spot price is set to $S_0 = 100$, and the initial variance is $V_0 = 0.0225$. The risk-free interest rate is $r = 5\%$, and the maturity is $T = 1$. The time interval $[0, T]$ is discretized into $N = 12$ time steps.
The Asian option has strike $K = 100$ and is monitored at the same dates as the hedging grid.
We consider proportional transaction costs with coefficient $\lambda = 0.001$.

Training Procedure: We compare a baseline method, where all controls are trained jointly, with the MMCC algorithm, which updates the controls sequentially backward in time.
For the MMCC method, we perform 10 outer iterations. At each iteration, 30,000 Monte Carlo paths are simulated and mini-batches of size 4096 are used. Each policy is trained for 300 gradient-based iterations using the Adam optimizer, and the initial control (y, q_0) is also trained for 300 iterations.
The baseline method is trained for 150 iterations.
All optimizations are performed using the Adam optimizer with learning rate 0.01.

Evaluation: The performance is evaluated on 100,000 independent Monte Carlo paths using the mean squared hedging error

$$\mathbb{E}\left[\left(Y_T - g(\bar{S}_T)\right)^2\right].$$

Method	Mean	Std dev	MSE	Q5%	Q50%	Q95%
MMCC	-0.0010	1.6603	2.7567	-2.9212	0.1544	2.3743
Baseline	-4.8352	1.8786	26.9080	-8.0807	-4.7043	-2.022

Table 1: Out-of-sample MMCC hedging performance.

Results: The results reported in Table 1 are obtained from the Python implementation of the algorithm. The MSE value achieved by the MMCC algorithm is 2.7567, and the mean error is -0.0010, which is close to zero, indicating an almost unbiased hedge. Moreover, the median hedging error is 0.1544, which is close to zero, suggesting that the trader is, on average, neither significantly over-hedged nor under-hedged.
The MSE is relatively low, reflecting a good hedging performance. However, due to the presence of transaction costs, perfect hedging is not achievable, and the market is therefore incomplete.

Figure 1 shows the convergence of the algorithms with respect to the number of iterations. The baseline method converges slowly and fails to reach a satisfactory MSE, whereas the MMCC algorithm rapidly decreases the error and stabilizes at a much lower level.
Figure 1 (right) highlights that the MMCC algorithm reaches an MSE below 3 after only 3 iterations, demonstrating a fast convergence.

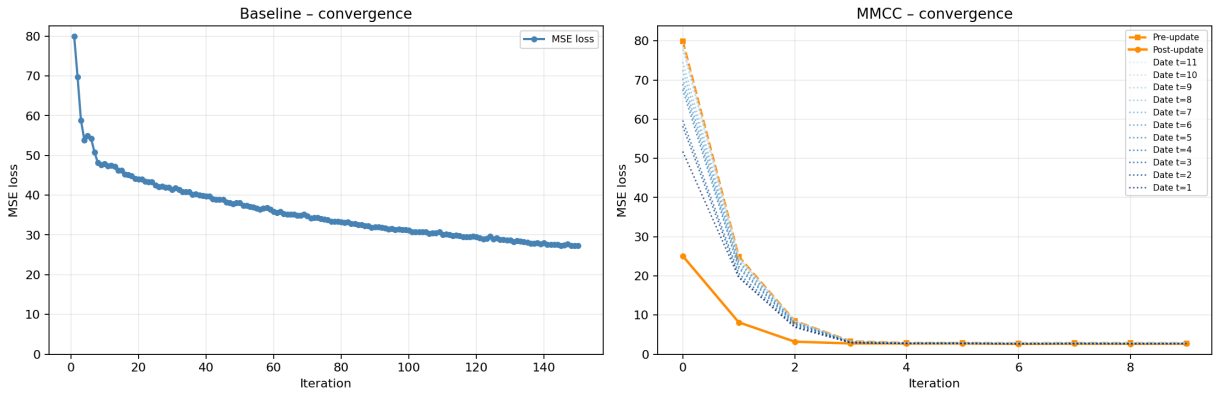

Figure 1: Convergence of the baseline (left) and MMCC (right) algorithms.

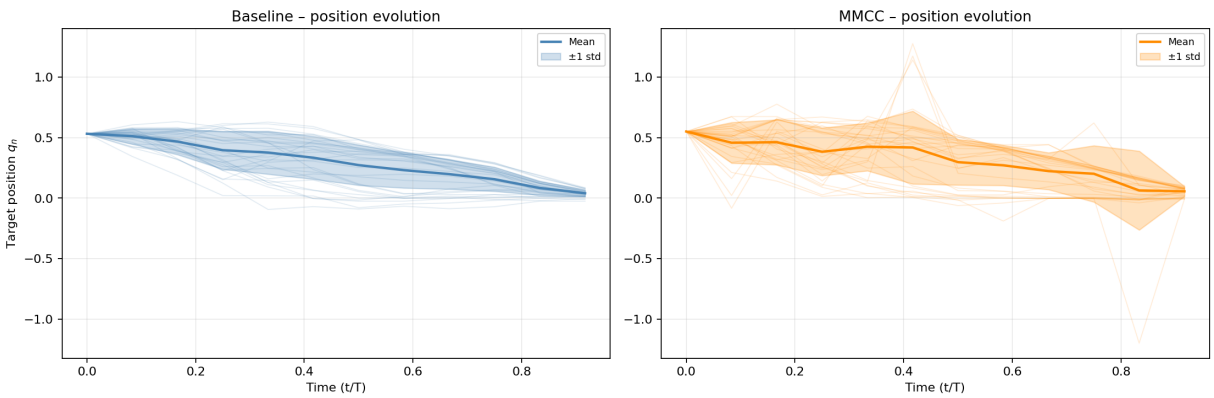

Figure 2: Evolution of the policy decisions (Q_{t_n}).

The value of y, corresponding to the initial option price (premium), obtained by the MMCC algorithm is 5.6799, and the initial position is $q_0 = 0.5512$.

Figure 2 shows the evolution of the hedging strategy Q_{t_n} over time. The baseline method (left) produces relatively smooth and homogeneous trajectories, which do not appear to adequately adapt to the evolution of the market. Such behaviour is not fully consistent with a dynamic hedging strategy. In contrast, the MMCC algorithm (right) exhibits more variability and responsiveness over time, suggesting a more realistic and adaptive hedging policy.

We also consider a second, more realistic configuration (Figure 3), corresponding to a one-month maturity with daily-rebalancing. This setting is closer to market practice, since in reality a trader would typically adjust the hedge at least daily on a liquid underlying. As expected, the overall mean squared hedging error is significantly smaller than in the previous experiments, because the higher rebalancing frequency allows the portfolio to track the option payoff more closely. Figure 3 shows that the qualitative convergence pattern remains similar to that observed in Figure 1. The MMCC algorithm reaches an MSE below 0.2 after only 3 iterations, demonstrating a fast convergence.

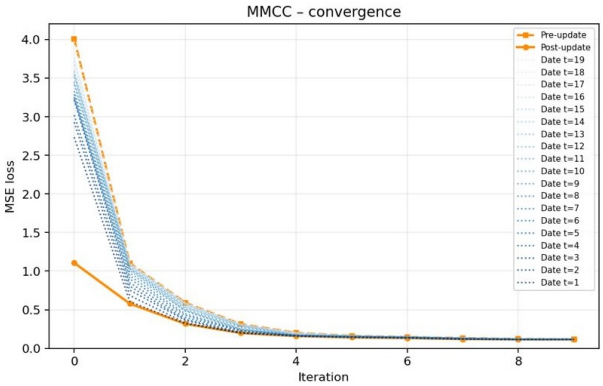

Figure 3: Convergence of the MMCC (T = 1 month).

Method	Mean	Std dev	MSE	Q5%	Q50%	Q95%
Baseline	−0.0767	0.5125	0.2685	−0.9647	−0.0341	0.6701
MMCC	0.0020	0.3368	0.1135	−0.5587	0.0102	0.5417

Table 2: Out-of-sample hedging performance comparison (T = 1 month).

Table 2 reports the hedging performance in the one-month, daily-rebalancing setting. The results show that MMCC achieves a very accurate hedge in this more realistic configuration. In particular, the mean error is 0.0020, which is essentially zero, indicating that the strategy is very well centered and almost unbiased on average. The MSE is also very small, equal to 0.1135, which reflects a good overall hedging performance.
Moreover, the standard deviation of the hedging error is only 0.3368, showing that the distribution of errors is fairly concentrated. The tail quantiles remain limited, with a 5% quantile of −0.5587 and a 95% quantile of 0.5417, while the median hedging error is 0.0102, again very close to zero. Overall, these figures indicate that MMCC produces a hedge that is both accurate and well centered, with a relatively tight distribution of outcomes. Compared with the baseline, the MMCC hedging error is also less dispersed.

6 Conclusion

Our implementation shows that the MMCC algorithm provides an effective solution to the considered problem. In particular, it is able to provide a coherent trajectory of date-wise hedging positions. The numerical results indicate a fast convergence of the algorithm together with a low out-of-sample hedging error, showing that the learned strategy provides an accurate hedge despite the presence of transaction costs and stochastic volatility. We also considered a more realistic short-maturity daily-rebalancing setting, in which the overall hedging errors become even smaller, while the qualitative behavior of the algorithm remains consistent.
Overall, these experiments confirm that MMCC provides a flexible and relevant framework for tackling finite-horizon stochastic control problems in finance, especially when path dependence, market frictions, and non-trivial state dynamics must be handled jointly.

References

[1] X. Peng, S. Kou and L. Zhang, *A Machine Learning Algorithm for Finite-Horizon Stochastic Control Problems in Economics*, 2024.

[2] Arnak Dalalyan, *Course Applied Statistical Learning, CREST, ENSAE Paris, IP Paris*, 2025.

[3] Steven L. Heston, *A Closed-Form Solution for Options with Stochastic Volatility with Applications to Bond and Currency Options*, 1993.